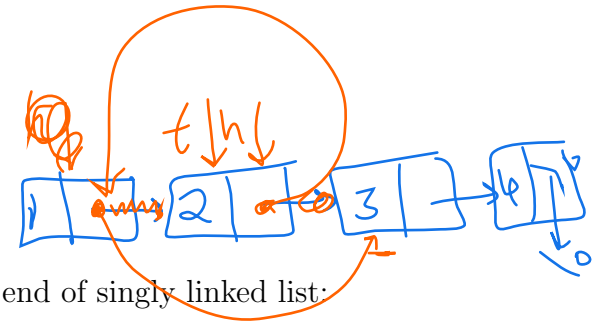
Practice Problems: Data Structures (Tracing & Code-Based)

1. What is final list after execution?

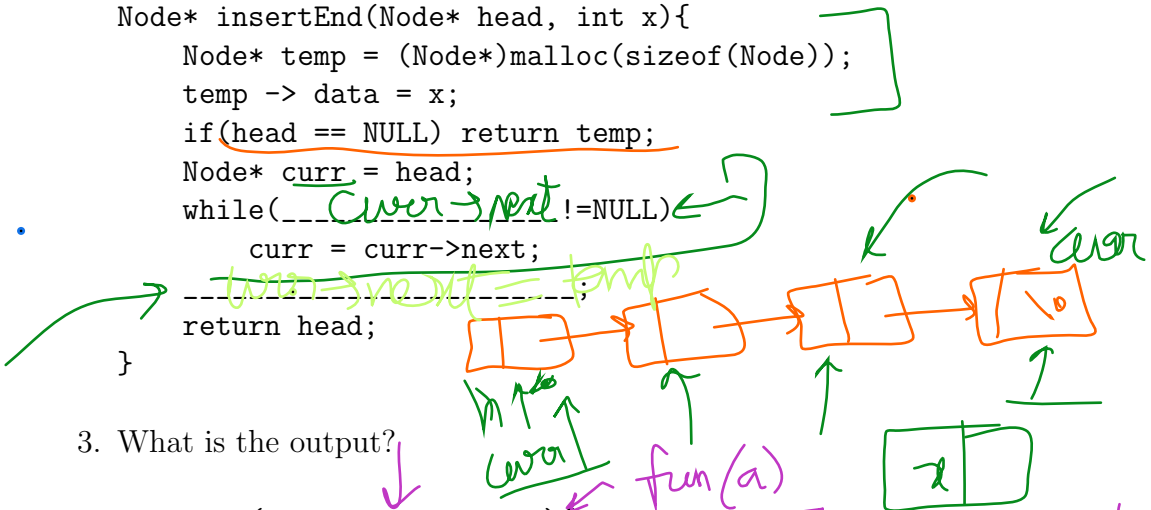
```
Node* temp = head->next;
head->next = temp->next;
temp->next = head;
head = temp;
```

Initial: $1 -> 2 -> 3 -> 4$



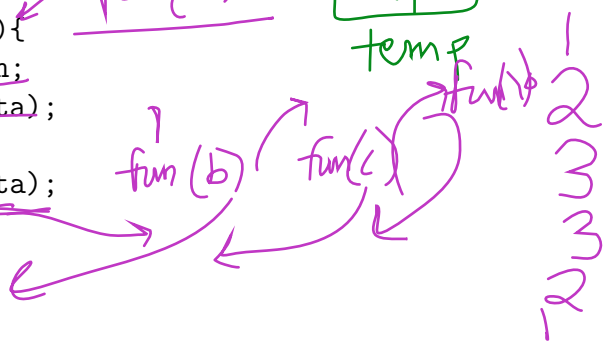
2. Complete the code to insert at end of singly linked list:

```
Node* insertEnd(Node* head, int x){
    Node* temp = (Node*)malloc(sizeof(Node));
    temp -> data = x;
    if(head == NULL) return temp;
    Node* curr = head;
    while(curr->next!=NULL)
        curr = curr->next;
    curr->next = temp;
    return head;
}
```



3. What is the output?

```
void fun(struct Node* head){
    if(head == NULL) return;
    printf("%d ", head->data);
    fun(head->next);
    printf("%d ", head->data);
}
```



For list: $1 -> 2 -> 3$

4. What is the output? [HW]

```
void fun(struct Node* head){
    if(head == NULL) return;
    if(head->next != NULL)
        printf("%d ", head->next->data);
    fun(head->next);
}
```

For list: $10 -> 20 -> 30 -> 40$

5. Predict output: [HW]

```
void fun(Node* head){
    if(head == NULL) return;
    fun(head->next);
    if(head->next != NULL)
        printf("%d ", head->next->data);
}
```

For: $1 -> 2 -> 3 -> 4$

6. Complete deletion of last node: [HW]

```
Node* deleteEnd(Node* head){
    if(head == NULL || head->next == NULL)
        return NULL;
    Node* curr = head;
    while(__________________)
        curr = curr->next;
    ________________________;
    return head;
}
```

7. Trace the function: [HW]

```
void fun(Node* head){
    if(head == NULL) return;
    printf("%d ", head->data);
    if(head->next != NULL)
        fun(head->next->next);
}
```

For list: $1 -> 2 -> 3 -> 4 -> 5$

8. Complete reverse (recursive): [HW]

```
Node* reverse(Node* head){
    if(head == NULL || head->next == NULL)
        return head;
    Node* rest = reverse(head->next);
    ___________________________;
    ___________________________;
    return rest;
}
```

9. Predict output: [HW]

```
void fun(Node* head){
    if(head == NULL) return;
    printf("%d ", head->data);
    fun(head->next->next);
}
```

For: $2 -> 4 -> 6 -> 8 -> 10$

10. **Assume the following structure:**

```
// Doubly Linked List
struct DNode {
    int data;
    struct DNode* prev;
    struct DNode* next;
};
```

Complete the code to insert an element at beginning in a doubly linked list.

```
struct DNode* insertAtBeg(struct DNode* head, int x, int pos) {
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct
        DNode));
    newNode->data = x;


        newNode->prev = NULL;
        newNode->next = ____________________;
        if(head != NULL)
            head->prev = ____________________;
        return newNode;

    return head;
}
```

11. Consider the following doubly linked list: [HW]

```
NULL <- [10] <-> [20] <-> [30] <-> [40] -> NULL
         ^                  ^
       head               ptr
```

Let ptr point to node with value 20.

We want to insert a new node with value 25 AFTER the node pointed by ptr.

Complete the following code snippet:

```
void insertAfter(struct Node* ptr, int x){
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = x;

    temp->next = _____________;

    temp->prev = _____________;

    _________ -> prev = temp;

    ptr -> next = _____________;
}
```

12. How can you detect whether a given linked list is circular or not? Provide a pseudocode for the function int $check\_if\_circular(node * head)$. [HW]

13. Stack operations. Show the stack after each operation. Push(10), Push(20), Push(30), Pop(), Push(40), Pop(), Pop()

14. Complete push:

```
void push(int st[], int *top, int x){
    if(*top == MAX-1) return;
    ____________________;
    ____________________;
}
```

15. Predict output: [HW]

```
int st[5], top = -1;
push(st,&top,5);
push(st,&top,10);
pop(st,&top);
push(st,&top,20);
printf("%d", st[top]);
```

16. Evaluate postfix (show stack after each step): 5 1 2 + 4 * + 3 -

17. Convert to postfix (trace stack): (A+B)*(C-D)/E

18. Queue operations. Show final queue: Enqueue(10), Enqueue(20), Dequeue(), Enqueue(30), Enqueue(40), Dequeue()

19. A **stack permutation** is a sequence obtained by pushing elements into a stack in a fixed order and popping them in some order, following the **LIFO (Last-In, First-Out)** rule. Not all permutations are possible because only the top element can be removed at any step.

    Given the input sequence $1, 2, 3, 4$, which of the following are valid stack permutations?

    (a) 2 1 4 3 *(b)* 3 1 4 2 *(c)* 2 3 4 1 *(d)* 4 3 2 1

20. Complete dequeue:

```
int dequeue(int q[], int *front, int *rear){
    if((*front > *rear)||(____________)) return -1;
    int val = q[*front];
    ____________________;
    return val;
}
```

21. Predict the final state of the queue:

```
void enqueue(int q[], int *rear, int x){
    (*rear)++;
    q[*rear] = x;
}
```

    Given:

    - Array size = 5
    - rear is initially = -1
    - The following operations are performed:

```
enqueue(q, &rear, 10);
enqueue(q, &rear, 20);
enqueue(q, &rear, 30);
```

    Questions:

    (a) What is the value of `rear` after these operations?

(b) What are the elements stored in the array?

(c) What limitation of this implementation can you observe?

22. Hash table size = 10, h(k)=k mod 10

    Insert: 23, 43, 13, 27 using linear probing. Show table after each insertion.

23. Same keys using quadratic probing: h(k,i) = (h(k) + i*i) mod 10

24. Double hashing: h1(k)=k mod 10, h2(k)=7-(k mod 7)

    Insert: 19, 29, 39. Show probe sequence.

25. Predict final table:

```
insert(10);
insert(20);
insert(30);
insert(40);
```

    Using h(k)=k mod 10 and linear probing.

26. Complete linear probing insert:

```
void insert(int hash[], int key){
    int i = key % 10;
    while(__________________)
        i = (i + 1) % 10;
    hash[i] = key;
}
```

27. Trace recursion stack:

```
int fun(int n){
    if(n == 0) return 0;
    return n + fun(n-1);
}
```

    For n = 3

28. Modify above to compute product instead of sum.